设计冻结 • v1.9 • 已冻结

orch 作为 Codex 的前台批处理层

目标：把 Codex 的额度花在"理解"和"交代"上，把"干活"外包给便宜算力

changkong34@qq.com ·2026 年 11 月 9 日 ·v1.9 办公工具链闭环 ·封存于 tag v1.9

> 冻结声明
>
> 本稿自 2026-11-09 起生效，对应代码 tag **v1.9**。 冻结的是四样东西：契约字段、命令面、错误码、验收标准。 不冻结的是"以后想加什么"——新增能力必须按 §8 的流程走，不许悄悄改。
>
> 判定依据不是"写完了"，而是：337 个测试全绿、四次真实对话端到端跑通、 任意一次运行可撤销、办公产物可生成且可断言。这四条都能在另一台机器上复现， 方法见 §8.1。

1 用户旅程

三种模式共用同一个开头和结尾，中间的执行形态不同。用户全程只做两件事： 在聊天窗口丢文件、说一句需求；最后看结论。

1.1 M1 单线（一个产出物）

```
用户：这批文件按模板统一格式（附 8 个文件）
Codex：① 读需求 → 写 intent  ② 调 orch dispatch  ③ 等结果  ④ 转述
orch：单任务 → 一个 worktree → 一个 worker → 验收 → 出产物
用户：（结果回到聊天，附产物）
```

适用：产出物唯一，或改动集中在一处。Codex 调用 1 次（阻塞返回）。

1.2 M2 并线（多个互不相关产出物）

```
用户：把这 12 篇稿子各自改成公众号版
Codex：① 写 intent  ② dispatch  ③ 轮询  ④ 汇总差异  ⑤ 转述
orch：拆 12 个任务 → 每任务一个 worktree → 并行执行 → 全绿后合并 → 产物
用户：（收到"12 篇完成，2 篇需人工看"）
```

适用：任务之间写入集互斥、无隐性依赖。Codex 调用 1 次；超过 8 分钟时加 1 次 `resume`。

1.3 M3 串联（后一步依赖前一步）

```
用户：先梳理这份报告的结构，再按结构重写
Codex：① 写 intent  ② dispatch  ③ 轮询  ④ 转述
orch：阶段1 出大纲 → handoff → 阶段2 按大纲重写 → 同一 worktree 内完成
用户：（收到最终稿 + 一份"大纲→成稿"的对应说明）
```

适用：后一步"必须看到前一步的结果才知道怎么做"。Codex 调用 1 次。

2　skill 的结构

skill 是路由表，不是说明书。Codex 照着走，不需要判断"该不该"。

```
~/.workbuddy/skills/orch-dispatch/
  SKILL.md              ← 一屏：前置检查 + 判断树 + 命令表
  references/
    contract.md         ← 契约字段与 26 条规则（出错时才读）
    errors.md           ← 报错码 → 动作 对照（出错时才读）
```

2.1　SKILL.md 的骨架

段落	内容
前置检查	自动批准模式是否开启 / key 是否可读 / 目标是否 git 仓库。任一不满足 → 直接告知用户，不继续
判断树	五条 if，输出三种模式之一（见 §3.1）
命令表	每种模式对应哪条 `orch` 命令、传什么参数
失败处理	遇到 E0xx 时读 `references/errors.md`，按表动作
禁止项	不得自行浏览仓库；不得修改用户文件；不得跳过 dispatch 直接改代码

> 一条硬规矩
>
> Codex 在 plan 阶段禁止主动探索仓库。它只允许读用户提供的文件与需求文本。 探索是 orch 的活——那部分走便宜算力，不烧额度。这条要写进"禁止项"，不能靠自觉。

2.4　规划分三层

规划不是一个模型干的活。它分三段，每段的成本结构完全不同——把强模型用在"理解"，把便宜模型用在"改格式"，这是省钱的关键。

阶段	谁做	输出	用强模型	为什么
初版计划	Codex	`tasks.json`	是	理解需求最吃模型能力，这一步值这个钱
校验	orch 规则	E0xx 错误清单	不需要	纯规则，零 token
修补	DeepSeek	修补后的计划	否	补锚点、拆任务这类机械改写，便宜模型够用

> 一条铁律：修补不能放宽标准
>
> 如果修补由便宜模型做，它最省力的路径是把判据改松来让自己通过。必须明令禁止，并由校验器核对修补前后的差异。

允许	禁止
补锚点（加文件名、数字、反引号符号）	删除或合并判据
把含糊判据改写成可执行命令	把命令型判据改成断言型
把一个任务拆成两个	扩大 `writes`

校验器在修补前后做对比，出现下列任一情况即拒绝：判据总数减少、任一任务的 `writes` 变大、某个任务失去全部命令型与锚点判据。

修补上限 2 轮，超过就退回 Codex，由它决定是补需求还是放弃。 无限修补会把"需求没写清"的成本，悄悄转成"模型反复改"的成本——而后者更贵且不可见。

Codex 在出计划时不能探索仓库（探索走便宜模型），所以初版计划必然缺上下文—— 这正是修补存在的理由，不是初版没写好。

2.5 执行单元有三层，不是两层

谈"换哪个模型"之前必须先分清三层。它们各自独立，混在一起谈就会得出错误结论。

层	决定什么	现状
壳 harness	能起什么进程、能不能多轮、沙箱边界	只有 `codex-cli` 在本机核验通过
模型 model	判断质量、花多少钱	三档，实测质量无差异
工具 tool	有没有确定性的转换与断言脚本	docx / xlsx / pptx 的内容层已覆盖（生成 + 断言）；版式层仍空

办公文档任务的瓶颈不在壳，在工具。换一个"更会写脚本的壳"， 换掉的只是"每次都要现写脚本"这件事的执行者，没换掉"每次都要现写"本身。

所以本设计给工具留了独立位置：壳跑不动的地方，由脚本跑——脚本不改主意， 两次结果一样，能被单测。注册表只为本机核验过的工具开放自动选中， 未核验的登记在册但不参与路由（重演过：未核验的档被兜底选中，失败却记成模型问题）。

3 契约增补

现有 9 个字段够用，补三个。

字段	归属	语义
`mode`	全局	`single` / `parallel` / `chain`。由 planner 提议，校验器复核
`inputs_from`	M3 / chain 必填	读哪个任务的产物。比 `deps` 更具体：`deps` 说"先后"，它说"读什么"
`handoff`	M3 运行时填	交接说明。默认内联 ≤200 字；超限自动转引用形态（见 §3.5）。不让完整对话进上下文
`integration`	全局，可选	集成级判据的来源文件（如 `.orch/verify.json`）。缺省则跳过集成验收

3.1 判断树（写进 skill）

第 0 步：要不要走这条线。Codex 有权拒绝调用编排，自己单干——见 §3.3。 判据有两个维度：互动类型（用户想干什么）与改动位置数（要动几处）。 不看行数——行数是伪精确：改 5 行横跨 3 个文件，风险高于改 40 行在同一函数内。

互动类型	判据	结论	理由
提问 / 解释	不产出文件，只回答问题	自干	没有产出物，编排无意义
单点修改	1 个文件 × 1 处连续位置	自干	用户能一眼验收，不需要机器判
探索	还不知道要改什么	自干一轮	探明之后再回到第 0 步重判
多点修改	位置数 ≥ 2（同文件多处也算）	走编排	需要逐点验收，人眼容易漏
批量同构	N ≥ 2 份同类产物	走编排	即使每份都简单，数量本身就是风险
跨文件改动	涉及 ≥ 2 个文件	走编排	需要确认改动之间的一致性

"一处"的定义：一个文件中的一个连续位置——一个函数、一个段落、一条配置项。 不是"一段 diff"，也不是"一片区域"。判据要在动手之前声明，不能改完了再补。

第 1 步：走哪条线。进入编排后，按下面的表选模式。

条件	模式	理由
需求只涉及一个产出物	single	没有可并行的部分
多个产出物，且彼此不读对方结果	parallel	并行收益最大
后一步必须看到前一步的结果才知道怎么做	chain	依赖是隐性的
多个产出物写同一文件	chain	并行必冲突，只能排队
单个产出物超一次上下文能装下的量	chain	必须分段处理

3.2 模式与计划的自洽性（新增三条规则）

编号	检查	动作
`E021`	声明 parallel，但写入集有交集	打回，要求改 chain 或拆任务
`E022`	声明 chain，但没有 `inputs_from`	打回，要求声明阶段依赖
`E023`	声明 single，但计划里任务数 > 1	打回，模式与计划不符

这三条的意义：让模型提议、让规则裁决。模式选错不会等到合并冲突才发现。

3.3 自干通道（Codex 有权拒绝）

Codex 可以判断"这活不值得走编排"，直接自己做——否则每个小活都要付一次编排的固定成本。

但这有个失败模式：自干最省事，而它烧的正是要省的额度。用三条约束框住。

约束	规则
默认编排	判据是"明确符合自干条件才自干"，不是"觉得麻烦就自干"。 错判的代价不对称：该编排却自干 → 额度白烧且没有验收；该自干却编排 → 只浪费几分钟。所以宁严不宽。
必须记账	每次自干往 `.orch/bypass.log` 追加一行：时间、互动类型、位置数、一句话摘要。未声明判据就动手的，视为违规。
必须交代	回复用户时说清"这次我自己做了，因为只有一处改动"。 不说明的话，用户无法判断是编排失败还是有意绕过。

> 为什么"必须记账"是硬要求
>
> 没有记账，就无法回答"自干率是否合理"。自干率过高说明额度没省下来；过低说明编排被滥用在小活上。 这个数字是产品是否健康的主要指标之一，而不是调试信息。

自干的产出仍然要满足同样的质量线——只是不建 worktree、不写契约、不跑机器验收。 省掉的是工装，不是正确性。

3.4 验收分两层

每个任务各有判据（任务级），但"各自通过 + 合并成功"不等于"拼起来能用"。需要第二层。

层	判据来源	回答什么
任务级	契约里每个任务的 `acceptance`	这一块改对了吗
集成级	仓库内固定文件（`.orch/verify.json`），worker 不可写	拼起来还能用吗

三种结果必须分开报，因为处置完全不同：

任务级	集成级	含义	处置
全绿	通过	真的成了	交付
全绿	失败	拆分有问题，不是某个 worker 的错	停，转重规划或人工
有红	不跑	有任务没做完	按失败处理

"任务绿但集成红"是这一层存在的全部理由。它是唯一能暴露"拆错了"的信号—— 只看任务级的话，这种失败会显示为成功。

为什么判据放固定文件而不是契约：契约由 planner 生成，planner 可能把标准写松； 固定文件由维护者维护并列入写集保护，worker 改不动。

3.5 handoff 溢出的降级

串联模式下，上一阶段给下一阶段的交接说明默认 200 字以内。 但当前一步产出的是几十页结构时，200 字装不下——强行压缩会把决策理由丢掉， 而决策理由恰恰是下一阶段最需要的东西。

所以设两种形态，超限自动切换，不截断：

形态	触发	存放	下一阶段看到
内联	≤ 200 字	契约字段 `handoff`	全文
引用	> 200 字	`.orch/handoff/<task>.md`，不提交	摘要 + 文件路径

三条规则：

1. 不截断。超限就转引用，截断会静默丢信息——比报错更危险。
2. 引用形态仍要 80–200 字摘要。只给路径等于让下一阶段从零读起，还不如不给。
3. 交接文件写进 worktree 但列入 **`.gitignore`**：越界检查豁免、合并不追踪、不进产物。

> 溢出次数是设计信号，不是运行日志
>
> 一个阶段需要写上千字交接，说明它本身该被拆成两段。 溢出次数记进 trace，连续出现就是阶段粒度过粗的证据——这是 M3 唯一的自校正信号。

3.6 任务入选标准：判据能不能写成一条命令

"文档类任务能不能做"这个问题没有统一答案，因为它其实是三种东西：

类型	例子	判据能写成命令吗	结论
文本内容	改写、摘要、润色、翻译	能：字数、禁用词、标题保留、术语覆盖	通（10 份实测 10/10）
结构操作	合并、拆分、抽表格、批量替换字段	能：段落数、字段值、序号连续	通
排版呈现	样式、页边距、图文混排、PPT 视觉	不能：只能"看着对"	不通

分界线不是文件后缀，是判据能不能写成一条没人看也能跑的断言。

> 这条标准同时是实验效度问题，不只是工程问题
>
> 判据弱 → "任务是否成功"这个因变量带噪声 → 拿它算成功率、算返工次数，结论不可用。 所以任务必须先过可断言性，再进实验集：写不出命令的案例，换案例，不是换判据。

实践上还有一条：实验用 **`.md`**，不要为了"文档感"去生成 **`.docx`**。 排版环节不可验收，把它拉进主实验只会污染数据。

确实要产出办公文件时，走确定性流水线而不是让模型自由发挥。 三种格式同构：模型只写内容，结构交给脚本，判据交给脚本。

产物	先写	再转（脚本）	再断言（脚本）
`.docx`	同名 `.md`	`docxgen`	`docxcheck --min-chars`
`.xlsx`	`.csv` 或含管道表的 `.md`	`xlsxgen`	`xlsxcheck --min-rows --cell A1=值`

产物	先写	再转（脚本）	再断言（脚本）
`.pptx`	`.md` 大纲	`pptxgen`	`pptxcheck --min-slides --require-title`

生成器写完会跑一次结构自检：内容类型是否覆盖每个部件、关系目标是否存在、 `r:id` 是否都有对应关系。这三样对不上，PowerPoint 只会说"无法读取"， 不告诉你哪儿错了——所以自检放在生成时，错误停在能看懂的地方。 判断标准也很清楚：写出来的包必须先通过自检，否则不许交付。

支持范围故意小，并且写进了工具说明与 worker 提示词： docx 只做标题/段落/列表/加粗；xlsx 只写单元格值；pptx 只做"一页标题 + 条目"。 图片、图表、配色、动画、条件格式都属于版式判断，仍属"看着对"那一类， 不塞进确定性流水线——宁可说做不到，也不假装做到了。

> 范围边界是设计的一部分，不是待办
>
> 流水线能覆盖的，是"内容对不对"；覆盖不了的，是"好不好看"。 把后者硬塞进来，只会让"验收通过"这句话重新变得不可信—— 而那正是这套东西唯一值钱的地方。

3.7 按写集后缀分类并记账

判断一个任务是代码还是文档，看 `writes` 的后缀就够了—— 它是任务必须精确声明的东西（写错会被 E013 拦），比让模型再判断一次既便宜又可靠。

类型	后缀	处理
代码	`.py .ts .js .go .rs .sql .html …`	现有路径
文档	`.md .txt .rst .docx .pdf`	现有路径 + 内容断言
表格 / 幻灯片	`.xlsx .csv` / `.pptx`	需要专门工具，未接则明确告警

分类结果进 trace（`run.kinds`）与报告（"任务构成"），它回答一个绕不开的问题： 我们跑过的活里，有多少是真能机械验收的。这个比例是"成功率"可信度的前提。

4 执行器接口

> 实测前提（2026-11-08）
>
> 外层 Codex 单次命令上限 ≥ 600 秒（实测跑满，用时 10 分 19 秒）。 所以轮询不必由 Codex 做——等待在脚本内部，Codex 只调一次。 这把"常驻宿主 + 消息投递"从必需品降为可选。

三个命令。默认同步阻塞，因为十秒到八分钟的批次占绝大多数。

命令	行为
`orch dispatch --repo <p> "需求原话" [--budget 480]`	规划 → 执行 → 验收 → 整合，阻塞直到完成，直接返回结论与产物清单。 `--budget` 是单次调用愿意等多久，到点未完成就返回进度与 `job_id`
`orch resume <id> [--budget 480]`	接着等。用在超过单次预算的长批次，通常一到两次就够

命令	行为
`orch undo <id>`	撤销：删该 job 的分支与 worktree，回到起点

状态落盘在 `runs/<job_id>/state.json`，不依赖常驻服务。 命令被中断也不丢状态，`resume` 接着跑。

4.1 阻塞还是投递：按批次时长分档

两种模式不是二选一，是分档。默认阻塞，因为绝大多数批次在 8 分钟以内。

1. 预计 ≤ 8 分钟 → 阻塞。一次调用拿到结果，对话被占用但可接受。
2. 8 – 16 分钟 → 阻塞 + `resume`。两次调用，对话被占两轮。
3. > 16 分钟，或用户说要离开 → 投递。起宿主，跑完把报告投回原会话。

为什么投递必须靠宿主：这个环境里，任何依附于命令的进程都活不过命令本身（`DETACHED_PROCESS` 被 Job Object 清掉、`BREAKAWAY` 被拒、`schtasks` 在黑名单）。 所以"命令秒回、任务继续跑"只能由对话之外启动的常驻进程承担。

档位不靠人猜，靠估。发起前先问一句 `orch eta --tasks N`： 它取历史 trace 里"每层墙钟"的中位数，乘以层数，得出预计时长。 超过 480 秒就转投递；宿主没启动则当场跑，并在输出里明说"可能被掐断"。 有外部计划时 `dispatch` 自己就会做这件事，不用人记得。

宿主的启动方式：开机自启，装一次永久生效，不要求用户双击脚本。 与"设环境变量"同属一次性安装配置。

4.2 卡死与回收

`resume` 的前提是状态可信。如果状态文件损坏、或 worker 成了僵尸，`resume` 会一直失败， 而进程仍占着内存和 worktree。没有回收机制，跑几次就会把机器拖死。

判活靠文件锁 + 心跳，不靠 pid——pid 会被复用，拿它判活等于赌运气； 而文件锁只能证明进程还活着，不能证明它在干活。两者要一起看。

锁文件里除了状态，必须写 `pid`、`run_id`、`last_updated`（心跳）和 `progress`（当前阶段与完成数）。

锁	心跳	结论	动作
被持有	< 30 分钟	真活着	不动
被持有	> 30 分钟	疑似挂起（进程在但没进展）	允许介入：需 `--force` 或人工确认后强杀
可获取	任意	进程已死（正常结束或崩溃）	正常结束只清残留；异常标 `aborted` 并回收

阈值默认 30 分钟，且不低于单任务超时的两倍（600 秒超时 → 至少 20 分钟），否则一次大上下文预填就会被误判成挂起。

心跳由 runner 的独立线程每 15 秒写一次——不能放在 worker 调用里，否则卡住时心跳也停。写入用「临时文件 + rename」。

命令	行为
`orch status --all`	列出所有 job：状态、锁是否被持有、占用多少 worktree 与磁盘
`orch cleanup <id>`	回收单个 job：删 worktree、分支，以及 `.orch/handoff/` 里该 run 的残留；保留 `runs/<id>/` 下的 trace、diff、验收证据
`orch cleanup --stale`	批量回收「标记 running 但锁可获取」的 job
`--force-kill`	强杀：见下

`--force-kill` 必须先校验再杀：按状态里的 pid 取出该进程的命令行， 确认其中含本 run 的 id，通过才 `taskkill /T /F`；校验不过拒绝执行。 否则 pid 复用时杀掉别人的进程，比不杀危险得多。

它只有两个入口：心跳超时（锁仍被持有但 `last_updated` 超过阈值）， 或用户明确要求终止。前者要带 `--force`，后者要带 run id 确认。 两种情况都记一条 `killed_by_force` 事件——事后统计"有多少次是被杀的"，比统计失败更有用。

`dispatch` 启动前自动跑一次 `cleanup --stale`： 脏 job 不回收会越积越多，等到磁盘满了才发现就晚了。

5 安全网

自动批准意味着没人看，以下六条是硬要求。

1. 全绿才合并—— 本层任务全部成功才合并；部分失败停住等指令。
2. 交付要过两层—— 任务级全绿 且 集成级通过，才算交付；两者分开报。
3. 越界即废—— 改动不在 `writes` 内 → 作废该次尝试（已有）。
4. 合并只进集成分支—— 不动用户工作区，产物由 Codex 落地。
5. 可撤销—— `undo` 能完整撤掉分支、worktree 与临时文件。
6. 崩溃可回收—— 锁可获取却标记 running 的 job，能一键判死并清理（§4.2）。

> 为什么"全绿才合并"是必须的
>
> 现在的实现是"成功的照常合并、失败的不合并"。人工确认时这没问题， 自动批准时它会看起来像跑完了——这是"看起来成功"的第四种形式， 前三种都在验收器里，这一种在合并层。

6 验收标准

四条硬指标，全过才算"能用"；过不了不许进大规模实验。

#	指标	判定
1	一次批处理，Codex 调用次数 ≤ 2	用日志统计，从"用户发消息"到"结果回到聊天"
2	10 份文档无人值守完成，失败可归因	每份失败必须给出错误码或一句原因，不能是"未知"
3	任意一次 run 可完整撤销	`undo` 后仓库回到起点，分支与 worktree 清空
4	自干率可解释	抽 10 次自干，互动类型与位置数均符合 §3.1，且有记账；自干率本身不作指标，只作观测

7 不做什么（划边界）

冻结的一部分是"明确不碰"，否则范围会一直膨胀。

- 不做 GUI——界面就是 Codex 窗口。
- 不做模型路由矩阵。worker 固定便宜档位——"按任务类型选模型"已被实验证伪。
- 不做多租户 / 权限体系。单人本机工具。
- 不做自动探索仓库。planner 只看用户给的东西。
- 冻结前不跑大规模实验——版本不固定，结论不可比。

8 冻结意味着什么

冻结之后，接口、字段、规则编号、验收标准都不再变。改任何一条，都要：

1. 记录"为什么上一版不够用"；
2. 重跑验收标准四条；
3. 重跑受影响的那批实验（不是重跑全部）。

冻结的是接口与验收标准，不是性能承诺。同样的输入，慢一点可以； 判据松一格不可以——那是这套东西唯一值钱的地方。

8.1 封存与复现

项	值
版本	`tag v1.9`（提交见 `git log -1`）
离线包	`dist/orch-v1.9.bundle`（单个文件，不依赖网络）
校验	`dist/SHA256SUMS.txt`
运行时	Python 3.13 • 纯标准库，无第三方依赖
外部程序	git 2.x；worker 需要 `codex` CLI（本机核验 0.167.0-alpha.3.2）
测试	337 个，约 8 分钟（大部分是真 git 操作，不是 mock）

换一台机器，三步：

1. `git clone dist/orch-v1.9.bundle orch && cd orch && git checkout v1.9`
2. `python -m unittest discover -s tests -t .` → 应输出 `Ran 337 tests ... OK`
3. `python -m orch --help` → 应看到 15 个子命令

不需要装任何包、不需要配数据库、不需要联网。worker 那一步才需要模型凭据， 而且它的失败会被明确报出来（缺 key、缺 git 身份、沙箱起不来），不会伪装成"模型做错了"。

这样大规模实验的结论才对得上号——数据是某个具体版本的属性，不是一个想法的属性。

版本记录

版本	改了什么	为什么
v1.0	初稿：三段用户旅程、skill 骨架、契约三个字段、四条安全网	把技术验证的结论固化成可实现的规格
v1.1	新增 §3.3 自干通道	不是所有活都值得走编排，Codex 需要拒绝权
v1.2	自干判据改为「互动类型 × 位置数」，删掉行数	行数是伪精确：改 5 行跨 3 个文件，比改 40 行同一函数更险
v1.3	执行器改为同步阻塞；删掉 Codex 侧轮询；宿主降为可选	实测外层命令上限 ≥ 600 秒，等待可以整个放进脚本
v1.4	验收分两层（任务级 / 集成级）；阻塞与投递按时长分档	各自通过不等于组合可用；投递通道已实测可用，宿主改为长批次专用
v1.5	handoff 超限转引用形态；新增 cleanup / force-kill	200 字装不下几十页结构，压缩会丢决策理由；崩溃后没有回收会拖死机器
v1.6	锁加心跳，超阈值允许强杀；cleanup 覆盖 handoff 残留	文件锁只能证明进程活着，证明不了它在干活
v1.7	规划分为初版（Codex）/ 校验（规则）/ 修补（DeepSeek），修补不得放宽标准	强模型只该用在"理解"，机械改写交给便宜模型；否则重写循环会把额度吃掉
v1.8	执行单元拆成壳/模型/工具三层；新增任务入选标准（可断言性）与按后缀分类记账	办公任务的瓶颈是缺确定性工具，不是缺更聪明的壳；弱判据会污染实验因变量
v1.9	docx / xlsx / pptx 三种格式的生成与断言全部落地，生成时自检包结构。本版封存	只有断言没有生成，文档任务是半条腿；而"生成成功"必须意味着"包是完整的"

每一版的改动都必须能回答"上一版哪里不够用"。答不上来的改动，不进冻结稿。